Incident handler's journal

Instructions

As you continue through this course, you may use this template to record your findings after completing an activity or to take notes on what you've learned about a specific tool or concept. You can also use this journal as a way to log the key takeaways about the different cybersecurity tools or concepts you encounter in this course.

Date: November 9, 2026	**Entry:** 1
Description	A small U.S. health care clinic was hit by ransomware from a phishing email that encrypted files and demanded payment for a decryption key. This entry is in the Detection and Analysis phase of the NIST Incident Response Lifecycle, because it records what was known after the incident was reported.
Tool(s) used	**None**
The 5 W's	**The 5 W's of the incident:** ● **Who** caused the incident? ● An organized group of unethical hackers, known to target the health care and transportation industries, sent the phishing emails. Their true identities are unknown. ● **What** happened? ● Targeted phishing emails with a malicious attachment were sent to several employees. Once an attachment was downloaded, it installed malware that gave the attackers access to the network. They then deployed ransomware that encrypted critical files, including patient records, and left a ransom note demanding

	payment for the decryption key. ● **When** did the incident occur? ● The incident happened on a Tuesday at about 9:00 a.m., when employees reported they couldn't access their files and saw a ransom note on their screens. ● **Where** did the incident happen? ● At a small U.S. health care clinic that provides primary-care services. The attack hit employees' computers and the company network, which shut down business operations. ● **Why** did the incident happen? ● The attackers got into the network with targeted phishing emails sent to several employees. The clinic likely had weak defenses against phishing, such as limited email filtering for malicious attachments or limited employee awareness training, which let the malware in. The attackers' goal was money: ransomware groups target health care because losing access to patient data puts heavy pressure on a clinic to pay.
Additional notes	Does the clinic train employees to spot phishing emails? Better awareness training and email filtering for malicious attachments could help prevent a repeat. Should the clinic pay the ransom, given there's no guarantee the attackers will hand over the decryption key?

Date: November 9, 2026	**Entry:** 2
Description	I used Wireshark in a course lab VM to inspect a 200-packet sample capture of a host browsing to opensource.google.com. This practice supports the Detection and Analysis phase of the NIST Incident Response Lifecycle, because packet inspection is how analysts examine network evidence.
Tool(s) used	I used Wireshark, a graphical network protocol analyzer, to open the course-provided sample.pcap file in a Windows lab VM. I applied display filters by IP address, MAC address, protocol, port, and payload text, including ip.addr == 142.250.1.139, udp.port == 53, tcp.port == 80, and tcp contains "curl". I inspected individual packets layer by layer (Frame, Ethernet II, IPv4, TCP/UDP, DNS, and HTTP). The filters showed a DNS response listing 142.250.1.139 first for opensource.google.com, a TCP SYN to port 80, and two HTTP GET / requests whose User-Agent was curl/7.74.0.
The 5 W's	This entry documents a packet analysis lab, not an incident investigation. The capture was provided by the course; no live network was monitored.
Additional notes	The capture held 200 packets, including SSH, DNS, ICMP, TCP, and HTTP. Searching payload text can show how traffic was generated, such as curl instead of a browser.

Date: November 9, 2026	**Entry:** 3
Description	I used tcpdump on a Linux lab VM to list interfaces, capture live traffic, and save port 80 packets to a file. This practice supports the Detection and Analysis phase of the NIST Incident Response Lifecycle, because command-line capture collects network evidence on a server.
Tool(s) used	I used tcpdump, a command-line packet analyzer, along with ifconfig and curl, in the course's Linux lab VM. After listing interfaces with sudo ifconfig and sudo tcpdump -D, I captured five live packets on eth0, then ran a background capture of nine TCP port 80 packets to capture.pcap. I generated the web traffic with curl opensource.google.com and read the saved file with tcpdump -nn -r in both verbose (-v) and hex/ASCII (-X) formats. The file showed a TCP three-way handshake between 172.17.0.2:37918 and 192.178.219.101:80, a GET / request from curl/7.88.1, and an HTTP 301 redirect to https://opensource.google/.
The 5 W's	This entry documents a packet capture lab, not an incident investigation. Only the lab VM's own traffic was captured; no outside network was monitored.
Additional notes	The -nn option turns off name lookups, which keeps the output accurate and avoids extra DNS queries during an investigation. The -w and -r options let an analyst save a capture and review it later.

Date: November 10, 2026	**Entry:** 4
Description	In a course scenario, I looked up a suspicious file hash in VirusTotal and mapped the indicators onto the Pyramid of Pain. This work is in the Detection and Analysis phase of the NIST Incident Response Lifecycle, because I confirmed the file was malicious and gathered related IoCs.
Tool(s) used	I used VirusTotal to look up the SHA256 hash 54e6ea47eb04634d3e87fd7787e2136ccfbcc80ade34f246a12cf93bab527f6b and reviewed the Detection, Details, Relations, and Behavior tabs. I also used MITRE ATT&CK to classify the sandbox behavior. 52 of 70 vendors flagged the file (bfsvc.exe) as a Flagpro/Fragtor trojan with a community score of -296. From the report I recorded the MD5 and SHA-1 hashes, the C2 domain org.misecure.com, an HTTP GET to http://org.misecure.com/index.html, and Credential Access techniques T1555, T1056, and T1539. I did not download or run the file.
The 5 W's	**The 5 W's of the incident:** ● **Who** caused the incident? ● In the course scenario, an unknown threat actor sent a phishing email with a password-protected spreadsheet attachment to an employee at a financial services company. I acted as a level-one SOC analyst reviewing the alert. ● **What** happened? ● The employee opened the attachment, which executed a malicious payload and created unauthorized executables on the computer. I looked up the SHA256 hash in VirusTotal and confirmed the file (bfsvc.exe) was malicious, then mapped the indicators of

	compromise onto the Pyramid of Pain. ● **When** did the incident occur? ● In the scenario, the email arrived at 1:11 p.m., the file was opened at 1:13 p.m., unauthorized executables appeared at 1:15 p.m., and the intrusion detection system alerted the SOC at 1:20 p.m. ● **Where** did the incident happen? ● On an employee's computer at a financial services company, after a file was downloaded from a phishing email. The hash lookup was done in VirusTotal. ● **Why** did the incident happen? ● The attachment was a credential-stealing trojan in the Flagpro family that contacts org.misecure.com for instructions, downloads secondary payloads, and exfiltrates stolen data over HTTP. Password-protecting the attachment helped it bypass email filters.
Additional notes	This was a course scenario; I only looked up the provided hash and did not download or run the malware. Blocking a hash is easy for defenders, but changing a file changes its hash, so indicators higher on the Pyramid of Pain, such as TTPs, cost the attacker more to change.

Date: November 10, 2026	**Entry:** 5
Description	I triaged ticket A-2703 with a phishing playbook, confirmed the attachment was malicious, and escalated it to a level-two SOC analyst. This work is in the Detection and Analysis phase of the NIST Incident Response Lifecycle, because I evaluated the alert and decided to escalate it.
Tool(s) used	I followed the course's phishing incident response playbook and used the VirusTotal verdict from my previous hash lookup. The playbook asked whether the email had attachments and whether they were malicious. Because bfsvc.exe was flagged by 52 of 70 vendors as the Flagpro trojan, I set the ticket status to Escalated and wrote comments for a level-two analyst.
The 5 W's	**The 5 W's of the incident:** ● **Who** caused the incident? ● An unknown threat actor posing as a job applicant named "Clyde West." The display name ("Def Communications"), the signature ("Clyde West"), and the random .su sender address (76tguyhh6tgftrt7tg.su, IP 114.114.114.114) don't match each other. ● **What** happened? ● A phishing email posing as a job application was sent to Inergy's HR inbox with a password-protected attachment, bfsvc.exe, whose hash is verified malicious. The alert severity was Medium, the recipient may have opened the file, and I escalated ticket A-2703 to a level-two SOC analyst. ● **When** did the incident occur? ● Wednesday, July 20, 2022, at 9:30:14 AM.

	● **Where** did the incident happen? ● Inergy's HR department inbox (hr@inergy.com, 176.157.125.93), flagged by the mail server alert. Inergy is a financial services company in the course scenario. ● **Why** did the incident happen? ● The attacker disguised the email as a resume, which HR expects to receive from strangers, and password-protected the file so email filters couldn't scan it. The message also had spelling and grammar errors, such as "Egnieer" and "Ingergy."
Additional notes	The ticket, email, and playbook were provided by the course and are fictional. I did not download or run the attachment. Password-protected attachments are a red flag because they stop email security tools from scanning what's inside.

Date: November 10, 2026	**Entry:** 6
Description	I ran a custom Suricata rule against a sample packet capture and analyzed the alerts in fast.log and eve.json with jq. This lab supports the Detection and Analysis phase of the NIST Incident Response Lifecycle, because an IDS helps analysts find suspicious traffic.
Tool(s) used	I used Suricata 7.0.10, an open-source IDS/IPS, and jq in the course's Linux lab to process sample.pcap with a custom rule (sid 12345, msg "GET on wire," rev 3). I ran sudo suricata -r sample.pcap -S custom.rules -k none, then read fast.log and eve.json. I used jq to pretty-print the JSON, extract fields such as timestamp, flow_id, signature, protocol, and dest_ip, and select all events for flow_id 695532541284239. Suricata read 200 packets and the rule fired twice, both HTTP GET requests from 172.21.224.2 to port 80 (142.250.1.139 and 142.250.1.102), with a severity of 3 and action allowed.
The 5 W's	This entry documents a signature and log analysis lab, not an incident investigation. The packet capture, rule file, and lab environment were provided by the course; no live network was monitored.
Additional notes	eve.json is the main log for incident response; fast.log is a deprecated one-line format useful for a quick check that a rule fired. Every network flow gets a unique flow_id, which ties together the events from the same connection.

Need another journal entry template?

If you want to add more journal entries, please copy one of the tables above and paste it into the template to use for future entries.

Reflections/Notes:

1. Were there any specific activities that were challenging for you? Why or why not?

The Suricata lab was the most challenging activity, mainly because of the jq commands used to read eve.json. My first compact field-extraction command left out jq itself, so the shell could not run it. A later flow_id query still had the literal X placeholder, and I had to fix the command before it returned the related events.

2. Has your understanding of incident detection and response changed since taking this course?

Yes, it has. I used to think detection and response was mostly about running tools and finding malware. This course showed me that clear documentation, the NIST Incident Response Lifecycle, and following a playbook are just as important, because they keep an investigation consistent and help the next analyst pick up the work.

3. Was there a specific tool or concept that you enjoyed the most? Why?

I enjoyed Suricata the most, especially using jq to query eve.json after I got the commands working. Seeing the custom "GET on wire" rule fire twice, then using a flow_id to pull every event from the same connection, showed me how an IDS log can tell a complete story of a network session.